

Assignment 07 Solutions: Binding, Polymorphism & Interfaces

Student Name: ahmed mohamed saed nofal

Course Code: ONL5_SWD5_S4

Group Name: ONL5_SWD5_S4

Topic: Static vs Dynamic Binding & C# Interfaces

Part 1: Static Binding (Early Binding) — Method Hiding with 'new'

Q1 & Q2. Implementation of Shape and Cube Classes

Here is the code for Shape and the derived class Cube using the new keyword to hide the base method:

```
// Base Class Shape
public class Shape
{
    public double Width { get; set; }
    public double Height { get; set; }

    public Shape(double width, double height)
    {
        Width = width;
        Height = height;
    }

    public double Area()
    {
        return Width * Height;
    }

    public override string ToString()
    {
        return $"(Width = {Width}, Height = {Height})";
    }
}

// Derived Class Cube
public class Cube : Shape
{
    public double Depth { get; set; }

    public Cube(double width, double height, double depth) : base(width, height)
    {
        Depth = depth;
    }

    // Hide Area() using new keyword
    public new double Area()
    {
        return base.Area() * Depth;
    }

    public void Print()
    {
        Console.WriteLine($"Width: {Width}, Height: {Height}, Depth: {Depth}");
    }
}
```

Q3. Main Method Execution & Static Binding Explanation

Output Results:

```
Shape shape = new Shape(2, 3); -> shape.Area() = 6  
Cube cube = new Cube(2, 3, 4); -> cube.Area() = 24  
Shape shapeRef = new Cube(2, 3, 4); -> shapeRef.Area() = 6
```

Explanation:

When we call `shapeRef.Area()`, it prints **6** (the Shape base version).

This happens because `Area()` in `Cube` uses the `new` keyword (method hiding). The compiler decides which method to call based on the **reference type** (`Shape`), not the actual object created in memory (`Cube`). This decision is made during compilation time, which is why it is called **Static Binding (Early Binding)**.

Q4. Object Reference and Polymorphic Behavior

When executing object `obj = new Cube(1, 2, 3);` and calling `obj.ToString()`, the output runs the `Shape` class version of `ToString()`.

Why did `ToString()` behave polymorphically while `Area()` did not?

- `ToString()` is declared as `virtual` in C#'s base `System.Object` class and was overridden using `override` in `Shape`. Override uses **Dynamic Binding (Late Binding)**, so the runtime checks the actual object type.

- `Area()` used the `new` keyword, which hides the base method instead of overriding it. Therefore, it follows Static Binding rules based on reference type.

Part 2: Dynamic Binding (Late Binding) — virtual / override

Q5 & Q6. Person, Doctor, and Engineer Classes

```
public class Person
{
    public int ID { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }

    public void Greet()
    {
        Console.WriteLine("I am a Person.");
    }

    public virtual void Display()
    {
        Console.WriteLine($"ID: {ID}, Name: {Name}, Age: {Age}");
    }
}

public class Doctor : Person
{
    public string Specialty { get; set; }

    public new void Greet()
    {
        Console.WriteLine("I am a Doctor.");
    }

    public override void Display()
    {
        Console.WriteLine($"ID: {ID}, Name: {Name}, Age: {Age}, Specialty: {Specialty}");
    }
}

public class Engineer : Person
{
    public string Field { get; set; }
    public int YearsOfExperience { get; set; }

    public new void Greet()
    {
        Console.WriteLine("I am an Engineer.");
    }

    public override void Display()
    {
        Console.WriteLine($"ID: {ID}, Name: {Name}, Age: {Age}, Field: {Field}, Experience: {YearsOfExperience} yrs");
    }
}
```

Q7. ProcessPerson Method & Method Resolution Analysis

ProcessPerson Code and Execution:

```
public static void ProcessPerson(Person person)
{
    person.Greet();
    person.Display();
}

// In Main:
Person doc = new Doctor { ID = 1, Name = "Dr. John", Age = 35, Specialty = "Cardiology" };
Person eng = new Engineer { ID = 2, Name = "Eng. Sarah", Age = 28, Field = "Software",
YearsOfExperience = 5 };

ProcessPerson(doc);
ProcessPerson(eng);
```

Predicted Output:

```
I am a Person.
ID: 1, Name: Dr. John, Age: 35, Specialty: Cardiology
I am a Person.
ID: 2, Name: Eng. Sarah, Age: 28, Field: Software, Experience: 5 yrs
```

Analysis:

- Greet() calls are resolved at **compile time** (Static Binding) because it is non-virtual. Since the parameter in ProcessPerson is of type Person, it always runs Person.Greet().
- Display() calls are resolved at **runtime** (Dynamic Binding) using the vtable because it is marked as virtual/override. It looks at the actual object created in memory (Doctor or Engineer).

Q8. Removing 'virtual' Keyword Consequences

If you remove virtual from Display() in Person, a **Compiler Warning** appears in the derived classes:
CS0114: 'Doctor.Display()' hides inherited member 'Person.Display()'. To make the current member override the inherited implementation, add the override keyword. Otherwise add the new keyword.

Meaning: You cannot use override if the base class method is not marked as virtual or abstract. The compiler alerts you that C# will default to method hiding (new) unless resolved.

Part 3: Interfaces

Q9. Pre-interface Problem

Problem: If we put all four methods (MoveForward, MoveBackward, MoveUp, MoveDown) in a single base class, every vehicle will inherit all four. This forces ground vehicles like Car or water vehicles like Ship to implement MoveUp() and MoveDown(), which makes no logical sense. Interfaces solve this by separating capabilities into modular contracts.

Q10 & Q11. IMoveable, IFlyable Interfaces and Class Implementation

```
public interface IMoveable
{
    void MoveForward();
    void MoveBackward();
}

public interface IFlyable
{
    void MoveUp();
    void MoveDown();
}

public class Car : IMoveable
{
    public void MoveForward() => Console.WriteLine("Car is driving forward on road.");
    public void MoveBackward() => Console.WriteLine("Car is reversing on road.");
}

public class Ship : IMoveable
{
    public void MoveForward() => Console.WriteLine("Ship is sailing forward on sea.");
    public void MoveBackward() => Console.WriteLine("Ship is reversing on sea.");
}

public class Airplane : IMoveable, IFlyable
{
    public void MoveForward() => Console.WriteLine("Airplane is flying forward in air.");
    public void MoveBackward() => Console.WriteLine("Airplane is backing up on tarmac.");
    public void MoveUp() => Console.WriteLine("Airplane is ascending into the sky.");
    public void MoveDown() => Console.WriteLine("Airplane is descending to land.");
}
```

Q12. Main Execution and Interface Reference Questions

```
IMoveable planeRef = new Airplane();
```

Can you call planeRef.MoveUp()?

No, you cannot. A compiler error occurs because planeRef is referenced as IMoveable, which only knows about MoveForward() and MoveBackward().

Required Reference Type: You must cast it or declare it using the IFlyable interface reference or concrete type Airplane:

```
IFlyable flyablePlane = (IFlyable)planeRef; flyablePlane.MoveUp();
```

Q13. Interface Inheritance (IVehicle)

```
public interface IVehicle : IMoveable, IFlyable { }

public class Vehicle : IVehicle
{
    public virtual void MoveForward() => Console.WriteLine("Vehicle move forward.");
    public virtual void MoveBackward() => Console.WriteLine("Vehicle move backward.");
    public virtual void MoveUp() => Console.WriteLine("Vehicle move up.");
    public virtual void MoveDown() => Console.WriteLine("Vehicle move down.");
}
```

Benefit of Interface Inheritance: It allows grouping multiple capability contracts into a single unified contract. Classes implementing IVehicle get both movement and flying requirements in one place without repeating interface lists.

Q14. Explicit Interface Implementation

```
public class Ship : IMoveable
{
    // Explicit Interface Implementation
    void IMoveable.MoveForward()
    {
        Console.WriteLine("Ship is moving forward (Explicit).");
    }

    public void MoveBackward()
    {
        Console.WriteLine("Ship is moving backward.");
    }
}
```

What happens when calling on Ship directly?

Writing `Ship myShip = new Ship(); myShip.MoveForward();` gives a compile error because explicit methods are private to the concrete class.

How to call it: You must call it through the interface reference:

```
IMoveable moveableShip = new Ship();moveableShip.MoveForward();
```

Part 4: Comprehension / Theory

Q15. Comparison Table

Feature	Static Binding (new)	Dynamic Binding (override)
Keyword in base	None (or regular method)	virtual / abstract
Keyword in derived	new	override
Resolved at	Compile time	Runtime
Behavior via base reference	Executes base class version	Executes derived class version

Q16. Why C# Requires 'virtual' before 'override'

In C#, methods are non-virtual by default for performance and safety.

- **virtual** tells C# to create a slot in the Virtual Method Table (vtable), explicitly granting permission for derived classes to modify its behavior.
- **override** connects the derived method to that exact vtable slot so polymorphism can happen at runtime.
- On the other hand, **new** does not modify the vtable; it simply creates a brand new method in the derived class that hides the base method name. Therefore, new can be used on any method without needing prior permission from the base class.